Exploratory Data Analysis (EDA)

What is EDA?

Exploratory Data Analysis is the process of examining datasets to:

- Summarize their main characteristics
- Detect patterns, trends, and relationships
- Identify anomalies or errors
- Inform data cleaning and later analysis decisions

EDA often uses **statistics and data visualization** to reveal what the data "looks like". It precedes modeling and helps shape hypotheses or feature engineering.

Goals and Components of EDA

There are three main **components of exploring data**:

1. **Understanding variables** (type, range, summary)
2. **Cleaning the dataset** (missing values, duplicates, errors)
3. **Analyzing relationships between variables** (correlation, distributions)

Python Libraries for EDA

- **Pandas** – data handling and manipulation
- **NumPy** – numerical operations
- **Matplotlib** – basic plotting
- **Seaborn** – statistical visualizations

Example import:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

These libraries make it easy to inspect and visualize datasets in Python.

Steps Involved in Data Exploration

1. Load and Inspect the Data

```
df = pd.read_csv("data.csv")    # Load dataset
```

Basic inspection functions:

```
df.shape          # Rows, columns
df.head()         # First 5 rows
df.tail()         # Last 5 rows
df.columns        # Column names
df.info()         # Data types and missing values
df.describe()     # Summary stats (mean, std, quartiles)
```

These give a quick overview of dataset structure and data types.

---

2. Understanding Variables

Unique values for categorical or discrete variables:

```
df['column'].unique()
```

Check value counts (important for categories):

```
df['category'].value_counts()
```

---

3. Data Cleaning (Wrangling)

a. Handling Missing or Null Values

Check missing values:

```
df.isnull().sum()
```

1) Delete missing values

```
df.dropna(inplace=True)
```

2) Impute missing values for continuous variables

```
df['age'].fillna(df['age'].mean(), inplace=True)
```

3) Impute missing values for categorical variables

```
df['gender'].fillna(df['gender'].mode()[0], inplace=True)
```

4) Other methods

- Use **forward fill** or **backward fill**

```
df.fillna(method='ffill', inplace=True)
```

- Predict missing values using models (advanced technique)

Note: Choose imputation method based on the business context and data distribution.

---

4. Removing Redundant Variables and Duplicates

Remove columns not needed:

```
df.drop(columns=['unnecessary_column'], inplace=True)
```

Remove duplicates:

```
df.drop_duplicates(inplace=True)
```

---

5. Detecting and Removing Outliers

Using Box Plot

```
sns.boxplot(x=df['column'])
plt.show()
```

Visual inspection reveals points outside the whiskers (possible outliers).

Using Z-Score

from scipy import stats

```
z_scores = np.abs(stats.zscore(df['column']))
df_outliers_removed = df[z_scores < 3]
```

Outliers are values beyond 3 standard deviations. Converting to a clean dataset improves quality.

---

6. Analyzing Relationships Between Variables

Correlation Matrix

```
corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.show()
```

- **Correlation Coefficient** values range from -1 to +1
  - +1: strong positive
  - -1: strong negative
  - 0: no linear relationship
    The heatmap visually helps understand which features move together.

---

7. Exploratory Visualizations

Histogram (Distribution)

```
df['column'].hist(bins=30)
plt.show()
```

Shows frequency distribution of values; helps identify skewness.

Scatter Plot (Relationship)

```
sns.scatterplot(x='feature1', y='feature2', data=df)
plt.show()
```

Used to explore relationships between numeric variables.

Box Plot (Outlier and Distribution)

```
sns.boxplot(x='column', data=df)
plt.show()
```

Helps detect outliers and shape of distribution.

Key Methods Used in EDA

Method	Description
df.shape	Number of rows & columns
df.head()	First few rows
df.columns	List column names
df.describe()	Summary statistics
df.unique()	Unique values in column
df.isnull().sum()	Count of missing values
Correlation matrix	Shows variable relationships
sns.scatterplot()	Plots two numeric variables
sns.histplot()	Helps see distribution
sns.boxplot()	Detects outliers

Data Wrangling (Preparation & Transformation)

What is Data Wrangling?

Data wrangling (data munging) is the process of:

- Cleaning and transforming raw data
- Making it usable for analysis
- Converting datasets into a structured format that enables analysis and visualization

It includes handling missing values, formatting data, reshaping data, and preparing for downstream tasks.

Key Data Wrangling Operations in Python

1. Filtering Data

```
df_filtered = df[df['column'] > 50]
```

2. Grouping Data

```
df_grouped = df.groupby('category')['value'].mean()
```

3. Reshaping Data

Using pivot:

```
df_pivot = df.pivot(index='date', columns='category', values='sales')
```

4. Sorting

```
df.sort_values(by='column', ascending=False)
```

5. Renaming Columns

```
df.rename(columns={'old': 'new'}, inplace=True)
```

6. Changing Data Types

```
df['column'] = df['column'].astype('float')
```

7. Handling Duplicates

```
df.drop_duplicates(inplace=True)
```

8. Merging and Joining

```
merged_df = pd.merge(df1, df2, on='key')
```

9. Converting Wide/Long Formats

```
df_melted = pd.melt(df, id_vars=['id'], value_vars=['col1', 'col2'])
```

These operations support cleaning, shaping, and preparing data for deeper analysis.

Complete Flow of EDA & Wrangling

1. **Load Data**
2. **Basic Inspection** (shape, info, head)
3. **Understand Variables** (unique, value_counts)
4. **Handle Missing Data** (dropna, fillna)
5. **Remove Redundancy** (drop, drop_duplicates)
6. **Detect and Handle Outliers** (boxplot, z-score)
7. **Summary Statistics** (describe)
8. **Variable Relationships** (correlation matrix, scatter plots)
9. **Visual Explorations** (histograms, countplots, boxplots)
10. **Wrangling Operations** (grouping, filtering, reshaping)
11. **Document insights and next steps**

Extra Tips for Practice

- Use .describe(include='all') to include categorical stats.
- Iterate by asking questions: “Why is this distribution skewed?”, “Do missing values follow a pattern?”
- Interpret graphs — look for peaks, gaps, skewness, and outliers.